

O3 Relearning Evaluation: Boundary-Shift 방식

1. 목적

기존 relearning switch 구현은 restore 대상 task의 OOD 계산을 건너뛰는 hard switch 방식이었다.

```
if task_name in restore_tasks:
    continue
```

이 방식은 구현이 단순하지만, OOD detector의 boundary를 조정한 것이 아니라 계산 경로를 제거한 것에 가깝다. 그래서 본 구현에서는 OOD detector를 그대로 사용하면서, restore 대상 task의 OOD score를 boundary 바깥쪽으로 이동시키는 **boundary-shift 방식**으로 수정했다.

핵심 목적은 다음과 같다.

- 추가 학습 없이 inference 단계에서 특정 task를 restore한다.
- OOD detector 계산은 유지한다.
- restore 대상 task의 soft LoRA weight만 낮춘다.
- restore하지 않은 task는 unlearned 상태로 유지한다.

수정 대상 파일은 다음 하나다.

```
ScienceQA_expriment/eval_o3.py
```

학습 코드(`train_ood.py` , `train_unlearn_lora_o.py`)와 checkpoint는 수정하지 않는다.

2. 원 논문 식과 아이디어

O3 논문에서 task t 에 대한 OOD boundary distance는 다음과 같이 표현된다.

$$d_{H^t}(x) = |s(x)^t - c^t| - R^t$$

여기서:

- x : 입력 query
- $s(x)^t$: task t 기준 feature 또는 score 위치
- c^t : task t 의 hypersphere 중심
- R^t : task t 의 hypersphere 반지름
- $d_{H^t}(x)$: 입력이 task t 의 forgotten knowledge 영역과 얼마나 관련 있는지를 나타내는 거리

우리가 원하는 것은 restore 대상 task에서 unlearning LoRA가 덜 적용되도록 만드는 것이다.

즉:

$$w_t(x) \downarrow$$

가 되도록 만들고 싶다.

이를 위해 restore 대상 task에 대해 effective boundary distance를 키운다. 원 논문 식에서 반지름 R^t 를 줄이는 것과 같은 효과를 생각하면:

$$\tilde{d}_{H^t}(x) = |s(x)^t - c^t| - (R^t - \Delta_t)$$

정리하면:

$$\tilde{d}_{H^t}(x) = d_{H^t}(x) + \Delta_t$$

즉, restore 대상 task에는 boundary distance를 추가로 더하는 효과가 생긴다.

Restore할 task 집합을 $\mathcal{R}$ 이라고 하면:

$$\Delta_t = \begin{cases} \delta, & t \in \mathcal{R} \\ 0, & t \notin \mathcal{R} \end{cases}$$

따라서:

$$\tilde{d}_{H^t}(x) = d_{H^t}(x) + \Delta_t$$

3. 공개 코드에서의 실제 흐름

공개 코드의 `eval_o3.py`에는 $s(x)^t$, c^t , R^t 가 명시적인 변수로 존재하지 않는다. 실제 inference에서는 다음 순서로 OOD weight가 계산된다.

```
mah_score = ood_models[i].get_unsup_Mah_score_s(
    ood_input,
    ood_mean_lists[i],
    ood_precision_lists[i],
    ood_fea_lists[i],
)[:, 1:]

test_score = ood_clrs[i].score_samples(mah_score)

w_ood = obtain_weights(test_score, ood_gmm_w_cls[i], ood_x0[i])
```

각 값의 의미는 다음과 같다.

1. mah_score

입력 query를 RoBERTa 기반 OOD model에 넣고, task별 feature 분포와 비교해 만든 Mahalanobis distance 계열의 중간 feature다.

2. test_score

task별 One-Class SVM(`ood_clrs[i]`)이 `mah_score`를 받아 계산한 OCSVM score다. 코드상에서 원 논문의 boundary distance 또는 OOD score에 가장 가까운 위치다.

3. `w_ood`

`test_score`, task별 GMM, `ood_x0[i]` 를 사용해 계산한 최종 OOD weight다. 이 값이 실제로 unlearning LoRA를 얼마나 적용할지 결정한다.

정리하면:

```
input x
-> mah_score: OOD 판단용 feature
-> test_score: OCSVM boundary 기준 score
-> w_ood: 최종 LoRA 적용 weight
```

따라서 boundary-shift를 적용할 위치는 `w_ood` 이후가 아니라, `test_score` 이후와 `obtain_weights()` 이전이다.

`w_ood` 를 직접 0으로 만들면 LoRA weight를 강제로 끄는 것에 가까워진다. 반면 `test_score` 를 조정하면 OOD detector의 boundary score를 수정한 뒤 기존 `obtain_weights()` 과정을 그대로 통과하게 된다.

4. Score-Space 근사 정의

공개 코드에 $s(x)^t$, c^t , R^t 가 직접 없으므로, OCSVM score 공간에서 대응되는 값을 정의한다.

$$z_t(x) = \text{OCSVM score}$$

코드 변수로는:

```
z_t(x) = test_score
```

그리고 score-space 기준점을:

$$b_t = x_0^t$$

라고 두면, 코드 변수로는:

```
b_t = ood_x0[i]
```

여기서 `ood_x0[i]` 는 원 논문의 c^t 자체가 아니다. OOD 학습 시 저장된 task별 score-space 기준점이며, 코드에서는 `test_score` 를 `w_ood` 로 변환할 때 중심점처럼 사용된다.

따라서 코드상 boundary distance를 다음처럼 정의한다.

$$d_t^{code}(x) = |z_t(x) - b_t|$$

restore 대상 task에 대해서는 이 거리에 margin Δ_t 를 더한다.

$$\tilde{d}_t^{code}(x) = d_t^{code}(x) + \Delta_t$$

하지만 `obtain_weights()` 는 distance가 아니라 score인 `test_score` 를 입력으로 받는다. 따라서 distance를 키운 뒤 다시 score 형태로 변환해야 한다.

원래 score가 기준점 b_t 보다 어느 방향에 있었는지를 유지하면:

$$\tilde{z}_t(x) = b_t + \text{sign}(z_t(x) - b_t) (|z_t(x) - b_t| + \Delta_t)$$

의미는 다음과 같다.

- 원래 `test_score` 가 `ood_x0[i]` 보다 크면 더 큰 쪽으로 이동한다.
- 원래 `test_score` 가 `ood_x0[i]` 보다 작으면 더 작은 쪽으로 이동한다.
- 즉, 기준점으로부터 더 멀어지게 만든다.
- 이는 원 논문 식에서 R^t 를 줄여 distance를 키우는 것과 유사한 효과를 낸다.

보고서에서는 다음처럼 표현하는 것이 안전하다.

We implement a boundary-distance shift in the OCSVM score space, which approximates the effect of reducing the task-specific hypersphere radius R^t .

한국어로는:

공개 코드에서는 R^t 가 직접적인 변수로 노출되어 있지 않기 때문에, OCSVM score 공간에서 boundary distance를 이동시키는 방식으로 R^t 감소와 유사한 효과를 구현한다.

5. 현재 구현

`eval_o3.py` 에 추가된 인자는 다음과 같다.

```
--restore_tasks biology physics
--restore_delta_scale 1.0
--output_file ./results/path.json
```

의미:

- `--restore_tasks`: boundary-shift를 적용할 restore 대상 task 목록
- `--restore_delta_scale`: boundary-shift margin의 scale
- `--output_file`: 결과 JSON 저장 경로

핵심 구현은 `test_score` 계산 직후에 들어간다.

```
test_score = ood_cls[i].score_samples(mah_score)

if i in restore_task_ids or ood_types[i] in restore_task_names:
    center = ood_x0[i]
    radius_proxy = abs(ood_x0[i] - ood_thresholds[i])
    delta = args.restore_delta_scale * radius_proxy
    direction = np.sign(test_score - center)
    direction = np.where(direction == 0, 1, direction)
```

```
test_score = center + direction * (np.abs(test_score - center) + delta)

w_ood = obtain_weights(test_score, ood_gmm_w_cls[i], ood_x0[i])
```

여기서:

$$\Delta_t = \lambda \cdot |x_0^t - \tau^t|$$

로 볼 수 있다.

- λ : `restore_delta_scale`
- x_0^t : `ood_x0[i]`
- τ^t : `ood_thresholds[i]`

`radius_proxy = abs(ood_x0[i] - ood_thresholds[i])` 를 쓰는 이유는 task마다 OCSVM score의 scale이 다르기 때문이다. 모든 task에 같은 raw delta를 더하는 것보다, task별 score scale에 맞춰 margin을 정규화하는 편이 안전하다.

6. 실행 예시

4개 task를 모두 restore하는 경우:

```
python eval_o3.py \
  --test_dataset ./data/scienceqa_SD_5/scienceqa_biology_physics_chemistry_economics_test_SD.json \
  --base_model gcyzsl/O3_LLAMA2_ScienceQA \
  --seed 0 \
  --lora_weights
./SCALE_0.1_seed_0_o_unlearn_lora_force_checkpoints_5/lora_force_random_biology_force_physics_force_chemistry_f
orce_economics_force \
  --ood_type _biology_physics_chemistry_economics \
  --ood_setting C \
  --ood_weights ./ood_checkpoints_scienceqa_0/ \
  --restore_tasks biology physics chemistry economics \
  --restore_delta_scale 2 \
  --output_file ./results_4task/boundary_shift_sweep/sd_test_delta2.json
```

특정 task만 restore하는 경우:

```
python eval_o3.py \
  --test_dataset ./data/scienceqa_SD_5/scienceqa_biology_test_SD.json \
  --base_model gcyzsl/O3_LLAMA2_ScienceQA \
  --seed 0 \
  --lora_weights
./SCALE_0.1_seed_0_o_unlearn_lora_force_checkpoints_5/lora_force_random_biology_force_physics_force_chemistry_f
orce_economics_force \
  --ood_type _biology_physics_chemistry_economics \
  --ood_setting C \
  --ood_weights ./ood_checkpoints_scienceqa_0/ \
```

```
--restore_tasks biology \
--restore_delta_scale 2 \
--output_file ./results_4task/task_restore_delta2/restore_biology/biology_test_SD.json
```

7. Sanity Check 결과

이 실험은 최종 보고서용 full experiment가 아니라, boundary-shift 방식이 실제로 작동하는지 확인하기 위한 sanity check다.

7.1 전체 restore

4-task unlearned 상태에서 target SD 성능은 약 26.14%였다.

Boundary-shift를 모든 target task에 적용하면 다음과 같이 복구되었다.

Setting	SD Test Acc.	RD Test Acc.	CommonQA Acc.	OpenBookQA Acc.
Boundary shift, delta 1	40.66	91.27	79.20	83.20
Boundary shift, delta 2	89.52	91.27	79.20	83.20

해석:

- delta 1 보다 delta 2 에서 target SD 성능이 크게 올라간다.
- RD, CommonQA, OpenBookQA 성능은 거의 유지된다.
- 따라서 restore strength를 restore_delta_scale 로 조절할 수 있음을 확인했다.

7.2 Task-wise restore

restore_delta_scale = 2 에서 restore task를 하나씩만 지정한 결과는 다음과 같다.

Restore W Test	Biology	Physics	Chemistry	Economics
Biology	99.24	25.52	26.89	34.52
Physics	23.93	68.75	26.89	34.52
Chemistry	23.93	25.52	87.39	34.52
Economics	23.93	25.52	26.89	94.05

해석:

- 대각선 성능만 크게 복구된다.
- restore하지 않은 task는 낮은 unlearned 성능으로 유지된다.
- 따라서 boundary-shift 방식이 전체 restore뿐 아니라 선택적 task restore에도 작동함을 확인했다.

8. 보고서용 요약 문장

영어:

We propose an inference-time relearning switch based on boundary-distance shifting. Instead of retraining the model or bypassing the OOD detector, we shift the OCSVM score of selected restore tasks away from the task-specific score-space reference point. This approximates increasing the effective boundary distance, reduces the corresponding soft LoRA weight, and reactivates the knowledge retained in the base model.

한국어:

본 프로젝트에서는 추가 학습 없이 inference 단계에서 특정 unlearning task를 복원하기 위한 boundary-shift 기반 relearning switch를 구현했다. 기존 hard switch처럼 OOD detector를 건너뛰는 대신, restore 대상 task의 OCSVM score를 task별 score-space 기준점으로부터 멀어지게 이동시켰다. 이는 effective boundary distance를 증가시키는 효과를 가지며, 해당 task의 soft LoRA weight를 낮춰 base model에 남아 있는 지식을 다시 활용하도록 한다.